GlassBox

Yassine Bourich
Youssef Bentaleb
Manal Zaidi
Farah Bouhzam

Project Vision

Every Decision in AutoML is Inspectable and Transparent

- **Zero dependencies,** NumPy-only.
- Designed for secure sandboxes and LLM agents.
- A Single entry point, autofit, returns a complete pipeline along with a comprehensive JSON report.

Monorepo Architecture

numpandas

Custom pandas clone built from NumPy.

EDA

Automated exploratory data analysis functionalities.

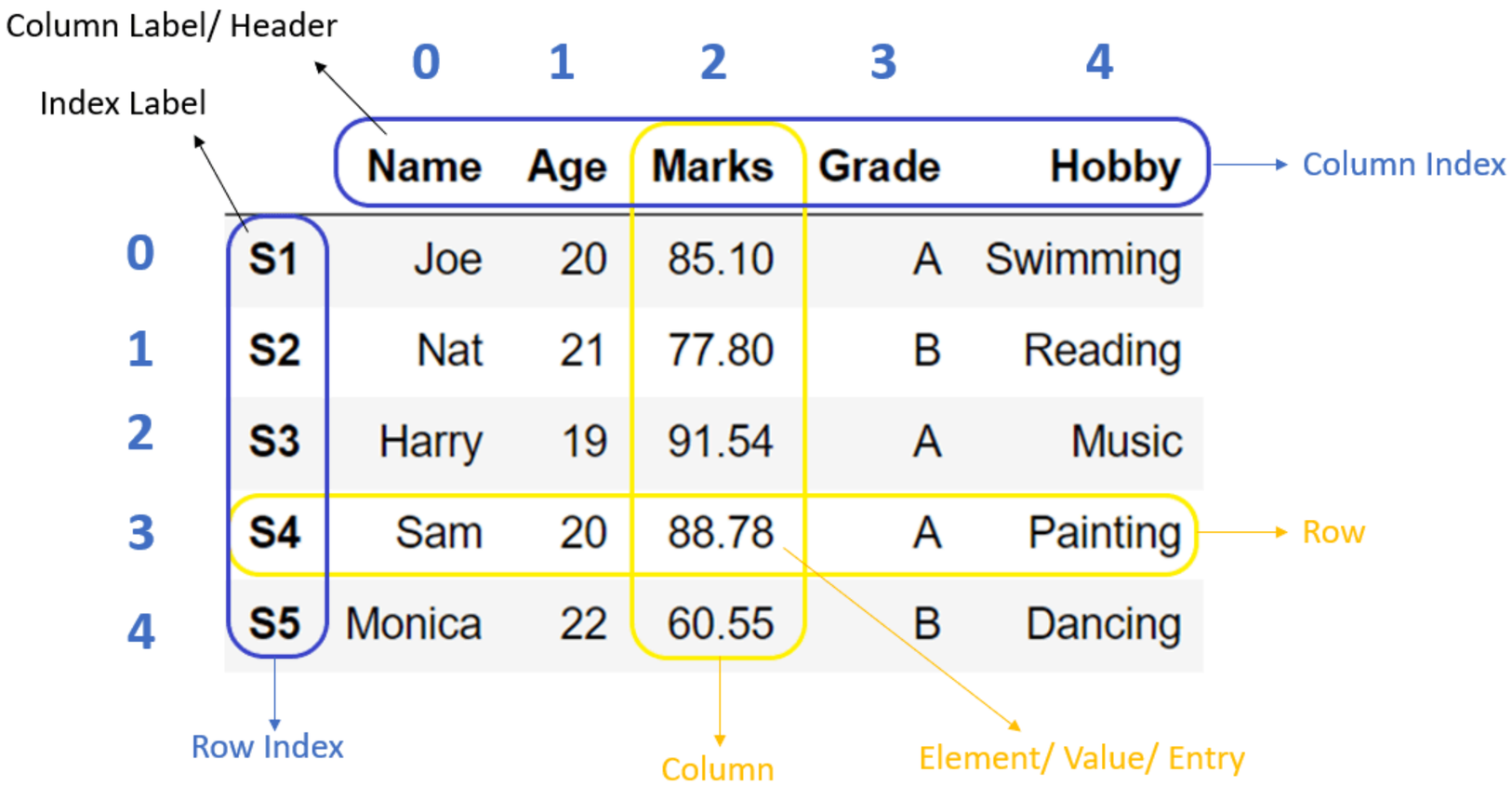
Preprocessing

Comprehensive data transformation and scaling components.

ML Models

Machine learning models, implemented from scratch.

Module 1: Numpandas Overview



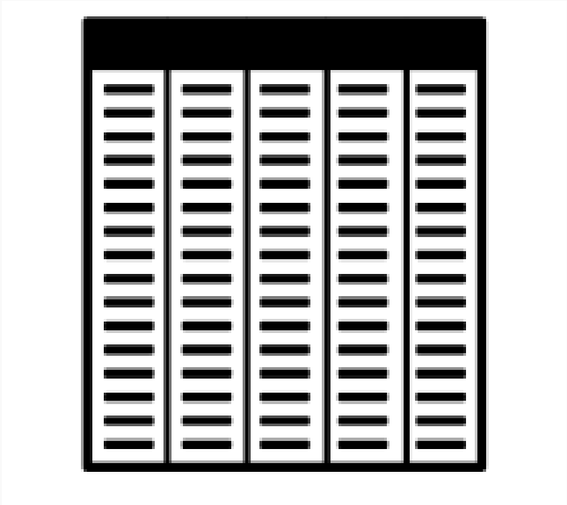
Custom Data Structures

Built from scratch using NumPy for flexibility.

Exploratory Data Analysis

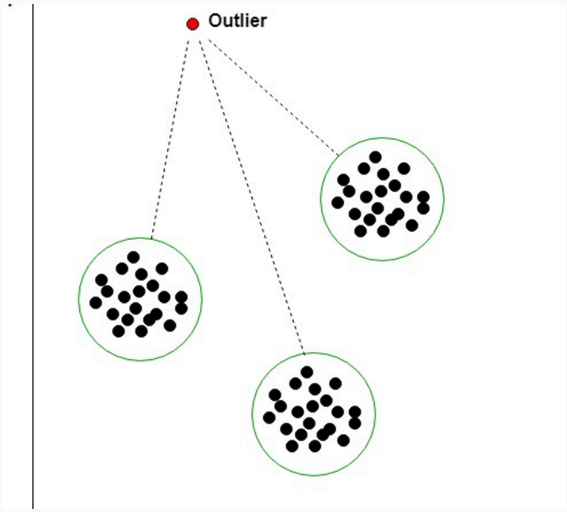
DataProfiler

Automates typing and statistical summaries efficiently



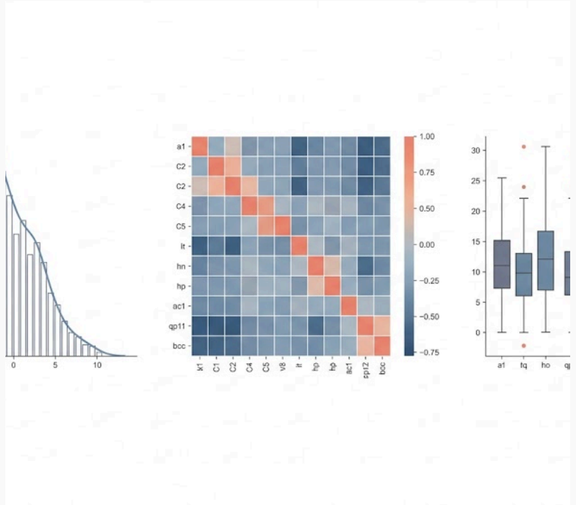
Outlier Detection

Identifies anomalies using statistical methods effectively



Statistical Visualizations

Histograms, boxplots, and scatterplots for insights



Exploratory Data Analysis

Plotters:

- HistPlotter
- BoxPlotter
- ScatterPlotter
- MissingnessPlotter
- CountPlotter
- CorrelationMatrixPlotter
- PairPlotMatrixPlotter

Profiler:

Generates an HTML documentation describing the data statistically.

Univariate Statistics:

- Mean
- Median
- Mode
- Standard deviation
- skewness
- kurtosis

IQR outliers detector

Data Transformation Components

Composing

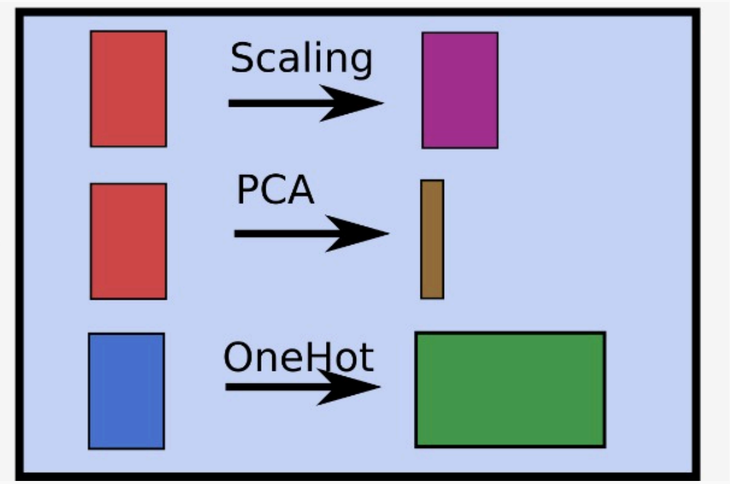
Apply appropriate and heterogeneous preprocessing techniques at specific features

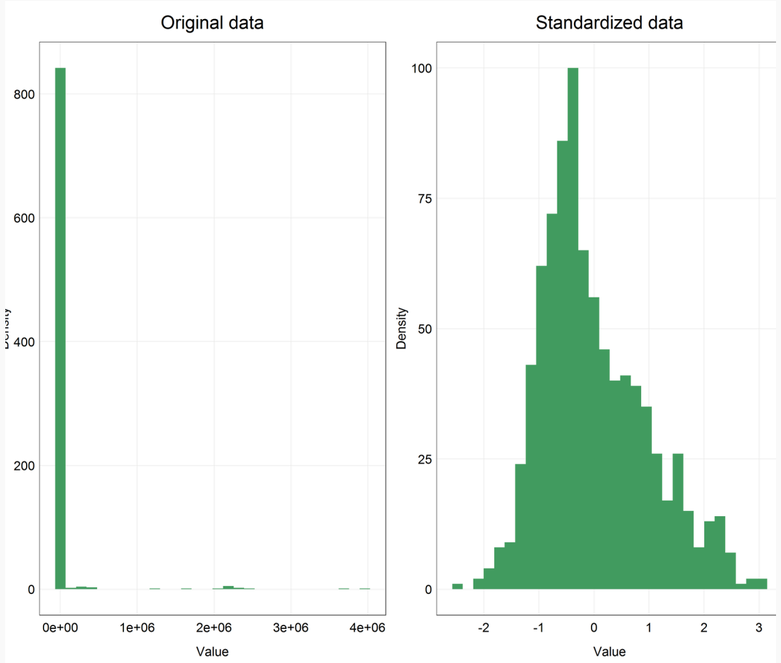
Scaling

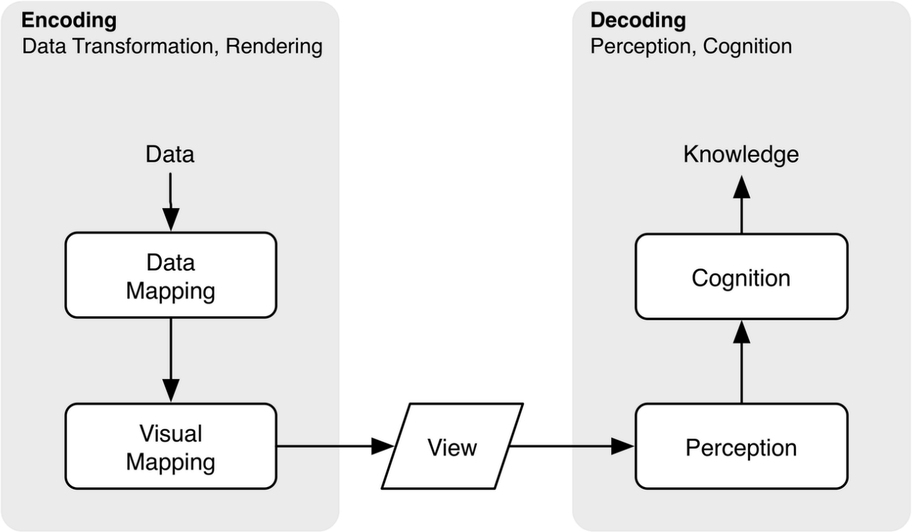
Normalizing data ranges for comparison

Encoding

Converting categorical data into numerical values



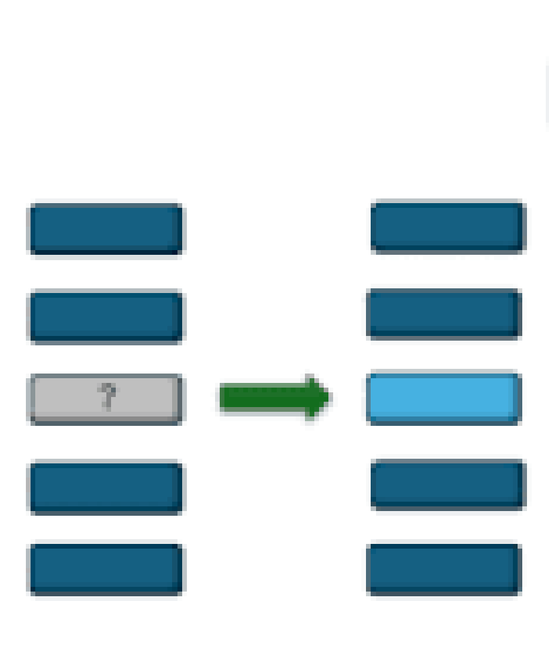




Data Transformation Components

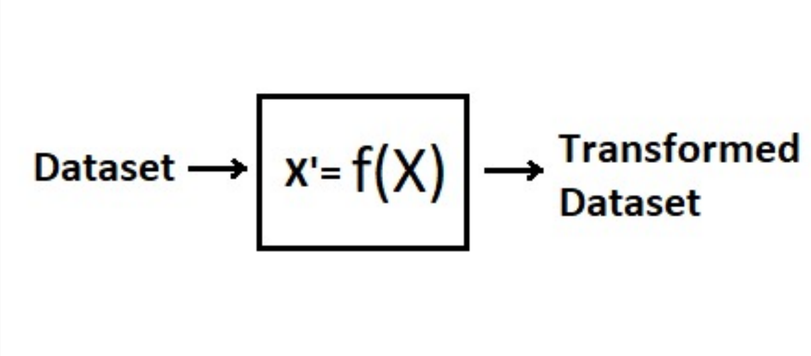
Imputation

Filling missing data with statistical methods



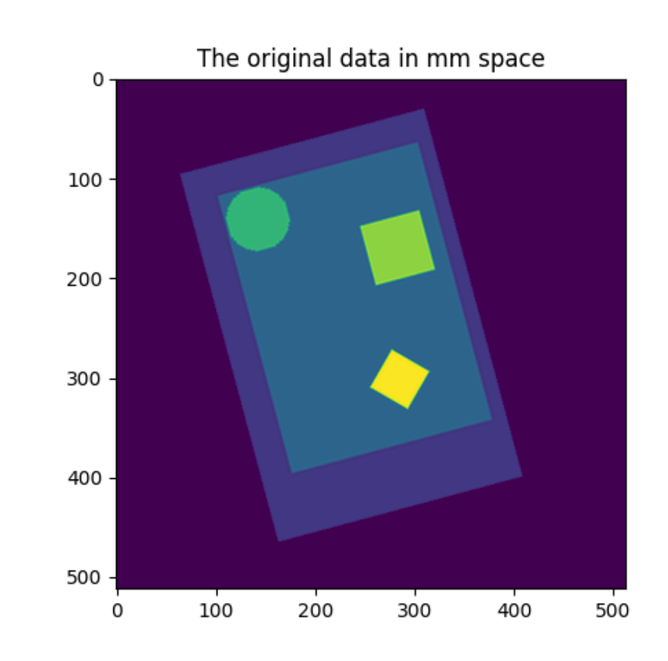
Function Transform

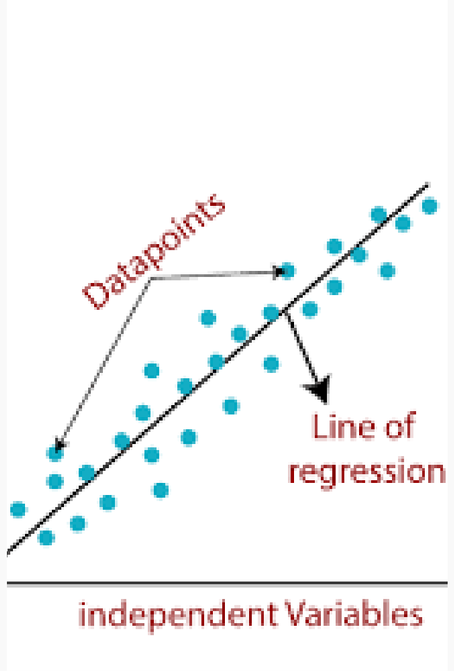
Apply feature engineering functions to the dataset to formulate new columns



Resampling

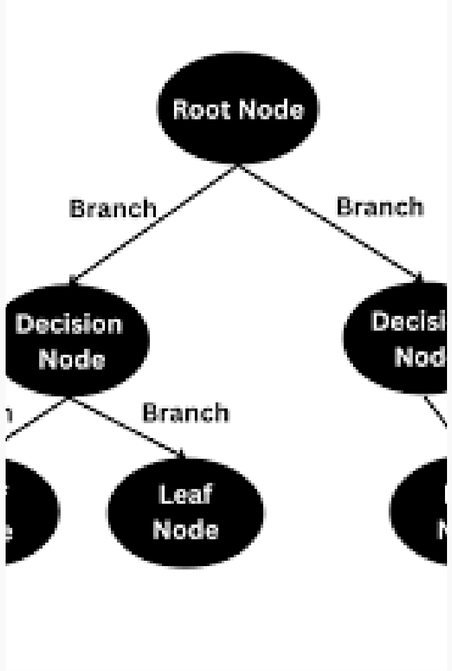
Balancing class distributions for analysis





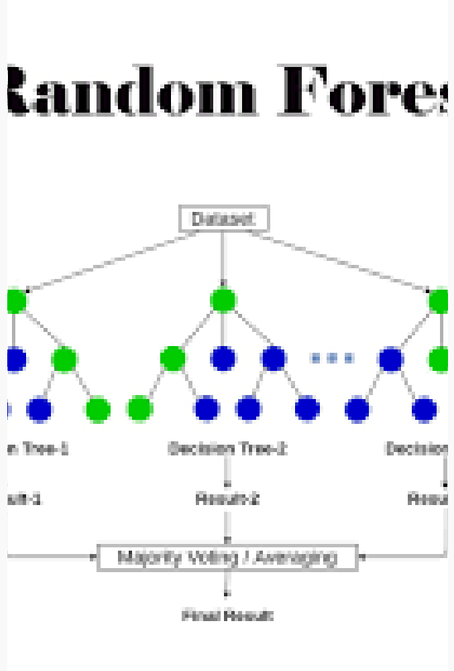
Regression

Predictive modeling for continuous outcomes



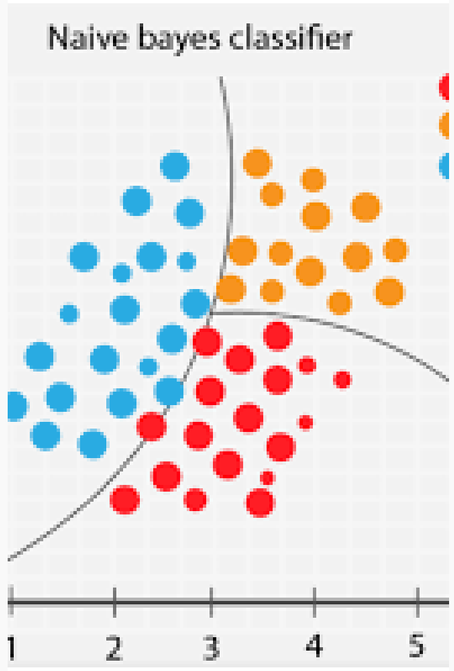
Decision Tree

Hierarchical structure for making decisions



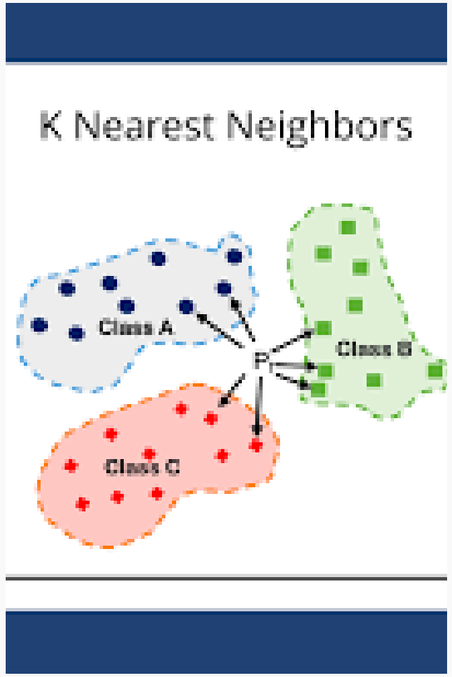
Random Forest

Ensemble learning through bagging techniques



Naive Bayes

Classification based on probabilistic reasoning



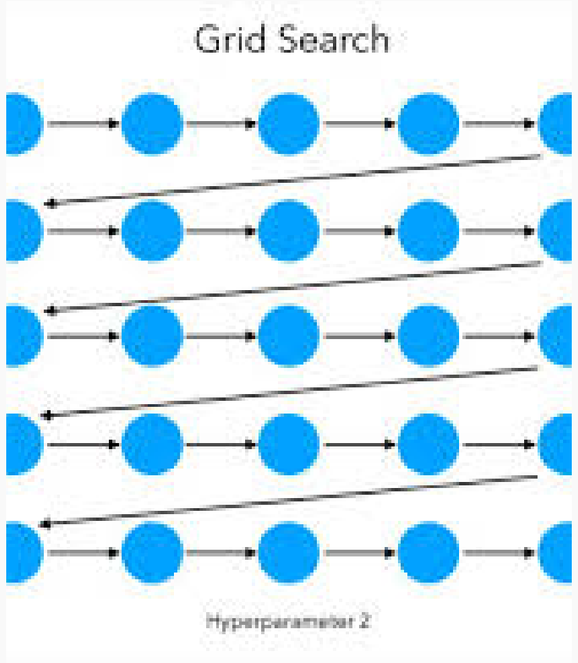
KNN

Proximity-based classification of data points

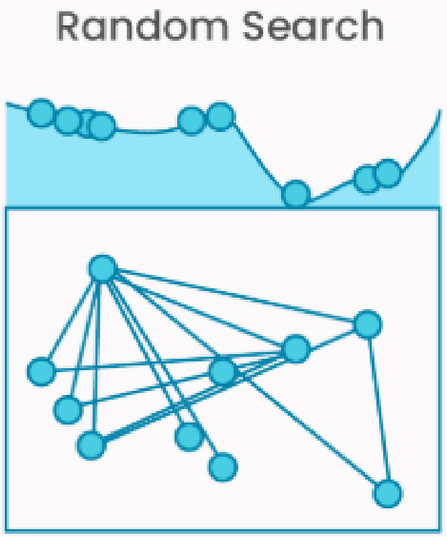
Key Machine Learning Models in GlassBox

Optimization Techniques Overview

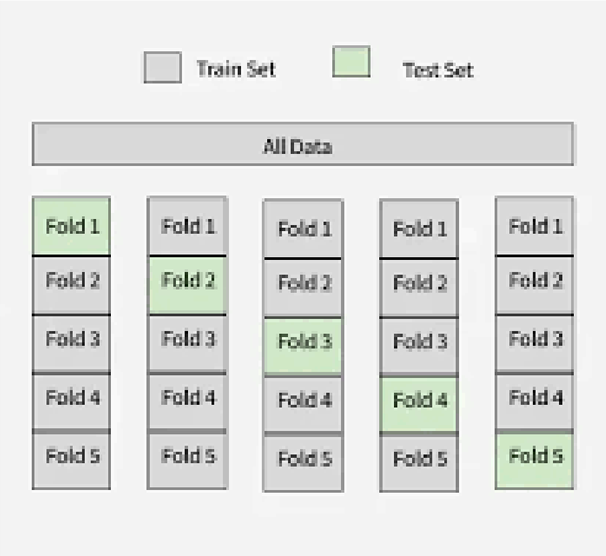
GridSearchCV
Exhaustive hyperparameter tuning across all models



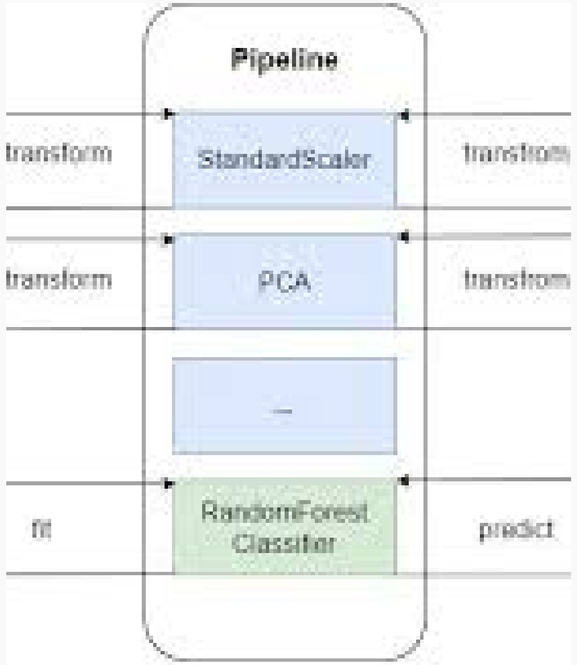
RandomizedSearchCV
Stochastic search with configurable iterations



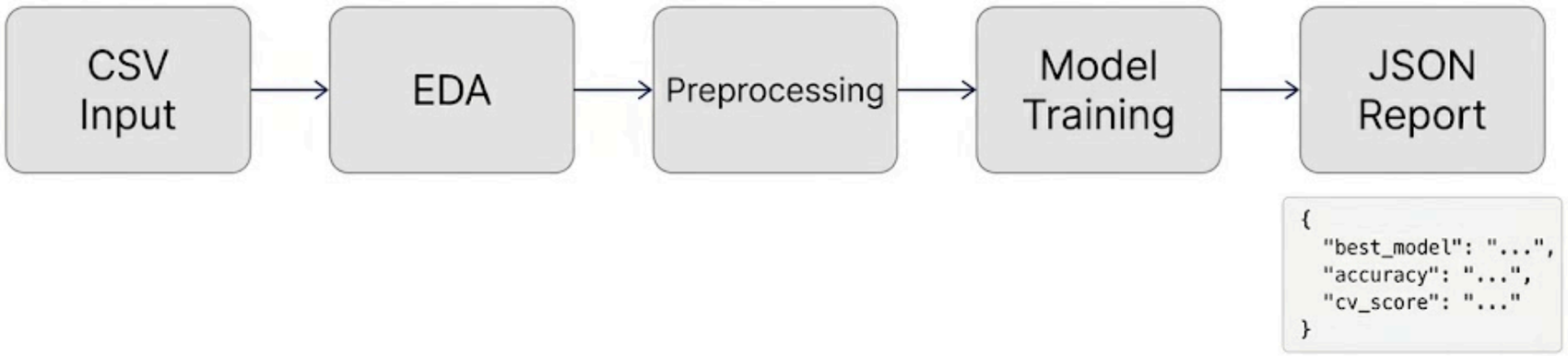
KFold CV
Custom splitter for cross-validation efficiency



Pipeline
Chainable workflows for seamless model integration



AutoFit: The Orchestrating Function



Full Pipeline

Orchestrates data loading, evaluation, and model training.

Comprehensive Analysis Output

Task Classification or Regression Insights

Exploratory Data Analysis

- Shape of dataset
- Data types overview
- Missing values detected
- Outliers identified
- Correlation analysis

Preprocessing Summary

- Imputers applied
- Scaling methods used
- Encoders implemented
- Resampling techniques

NemoClaw Agent Integration

Tool for LLM Agents with Actionable Insights

Securing GlassBox: Dockerization & NemoClaw AI Integration

The challenge

Allowing an AI agent to execute complex, multi-package Python scripts without risking the host machine's security or encountering local dependency conflicts.

The solution

A dual-layer infrastructure utilizing Docker for environment control and NVIDIA's NemoClaw for secure agent orchestration.

The result

A fully isolated, cross-platform pipeline where the LLM brain lives in the cloud, but code execution happens strictly inside a locked-down local vault.

Building the Container

Dockerfile

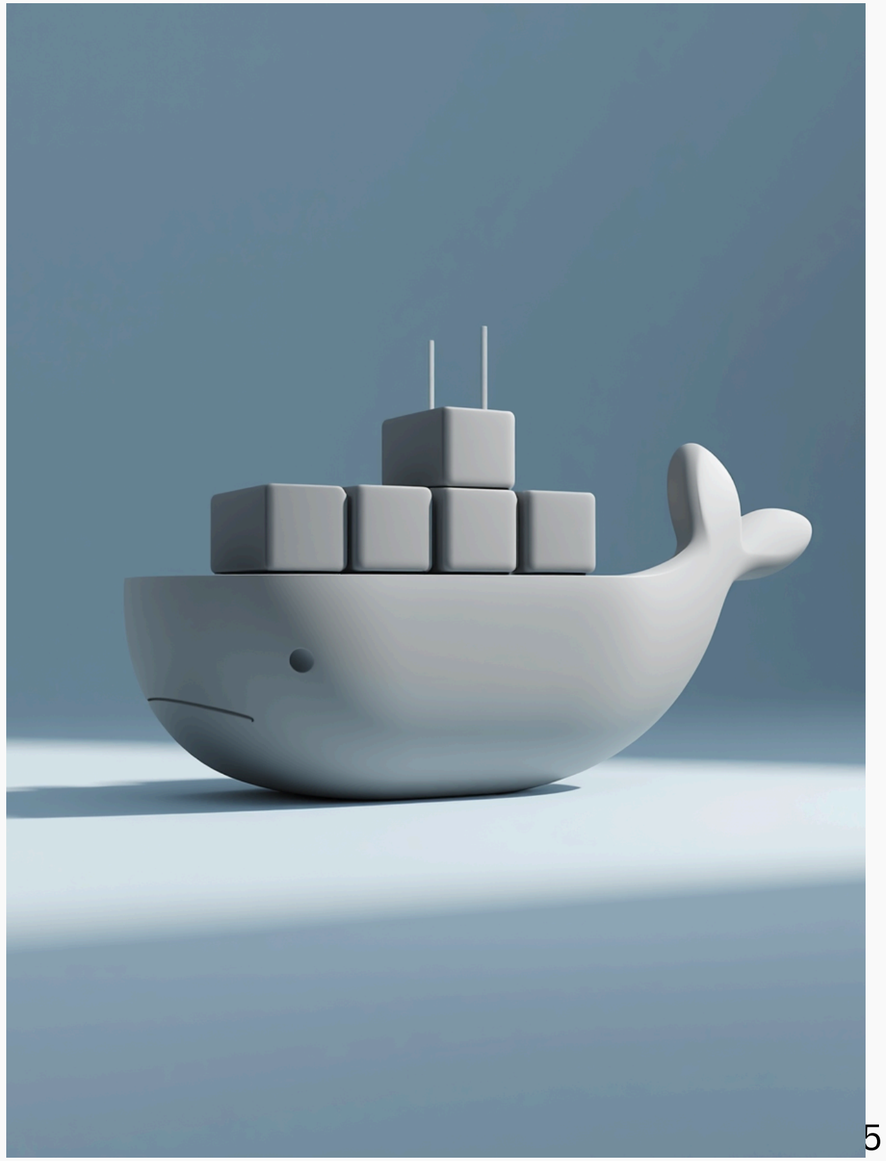
- A custom Dockerfile handles environment setup: non-root user, I/O mount points, dependency install, and PYTHONPATH injection across all 10 internal packages

Build:

```
docker build -t glassbox-env:latest .
```

Run:

```
docker run --rm \
  -v "${PWD}\test_data:/data" \
  -v "${PWD}\results:/results" \
  glassbox-env:latest \
  python -m GlassBox.autofit.cli \
    --data /data/test_model.csv \
    --target target \
    --output /results/best_model.pkl
```



NemoClaw installation

```
curl -fsSL https://www.nvidia.com/nemoclaw.sh | bash

nemoclaw onboard
```

One-Command Onboarding:

- The official nemoclaw.sh script handles everything inside the Linux subsystem with no manual configuration

Choosing the Brain:

- Integrated Google AI Studio with Gemini 2.5 Flash, reliable with complex JSON tool and has strict instruction-following under multi-step agentic workflows

NemoClaw & the OpenShell Security Architecture

The orchestration layer that keeps the agent honest

Deny-by-Default Posture:

- Complete isolation from the public internet.
- Zero-trust posture: No API calls or data exfiltration allowed unless explicitly whitelisted by a security policy.

Command Interception:

- The Agent never interacts with the Host OS.
- NemoClaw intercepts subprocess calls and redirects them into the Docker vault, keeping the "Brain" separated from the "Hardware."

Policy Tiers:

- Granular control over tool access (e.g., restricted vs. open).
- Ensures the agent has exactly enough permission to work, but never enough to damage.

=> Even if the LLM hallucinates a malicious command, the sandbox ensures it goes nowhere

The Agent Bridge

How to make the LLM talk to the container?

Tool Schema:

- glassbox_tool.py translates Docker CLI arguments (--data, --target, --output) into a structured JSON schema the LLM can natively reason about and invoke

The payoff:

- One natural language prompt : *"Analyze test_model.csv and predict the target column"* triggers the full pipeline:

schema parsing → Docker execution → JSON report → serialized model saved to results/

Final result

Run python run_agent.py inside agent/ directory and prompt the assistant:

```
cd agent
python run_agent.py
```

- The agent tool asks for a prompt
- Agent mounts both test_data and results, and persists the serialized model to results/best_model.pkl on the host.

Benchmarking: Comparison between GlassBox and Pandas/Scikit-learn

Group	Case	GlassBox seconds	pandas/sklearn seconds	Speedup	Winner
DataFrame	mean 5000x8	0.000267	0.000188	0.70x	pandas/sklearn
DataFrame	sum 5000x8	0.000148	0.000345	2.33x	GlassBox
Metrics	accuracy_score n=5000	0.000028	0.000518	18.41x	GlassBox
Metrics	mean_squared_error n=5000	0.000031	0.000208	6.72x	GlassBox
Models	KNN predict 1200x8	0.037490	0.003705	0.10x	pandas/sklearn
Models	linear regression fit 5000x8	0.015773	0.004097	0.26x	pandas/sklearn
Models	logistic regression fit 5000x8	0.351396	0.006279	0.02x	pandas/sklearn
Preprocessing	MinMaxScaler 5000x8	0.002362	0.000692	0.29x	pandas/sklearn

Benchmarking: Comparison between GlassBox and Pandas/Scikit-learn

Group	Case	Speedup	Winner
Metrics	accuracy_score n=5000	18.41x	GlassBox
Metrics	mean_squared_error n=5000	6.72x	GlassBox
DataFrame	sum 5000x8	2.33x	GlassBox
Models	logistic regression fit 5000x8	0.02x	pandas/sklearn
Models	KNN predict 1200x8	0.10x	pandas/sklearn

Benchmarking: Comparison between GlassBox and Pandas/Scikit-learn

Correctness And Quality

Group	Case	Library	Metric	Value
DataFrame	mean 5000x8	GlassBox	max_abs_diff	0
DataFrame	sum 5000x8	GlassBox	max_abs_diff	0
Preprocessing	MinMaxScaler 5000x8	GlassBox	max_abs_diff	2.220446049e-16
Metrics	accuracy_score n=5000	GlassBox	abs_diff	0
Metrics	mean_squared_error n=5000	GlassBox	abs_diff	0
Models	linear regression fit 5000x8	GlassBox	r2	0.9985625352
Models	linear regression fit 5000x8	pandas/sklearn	r2	0.9985625351
Models	logistic regression fit 5000x8	GlassBox	accuracy	0.9944
Models	logistic regression fit 5000x8	pandas/sklearn	accuracy	0.9944
Models	KNN predict 1200x8	GlassBox	accuracy	0.88
Models	KNN predict 1200x8	pandas/sklearn	accuracy	0.88

90% of Accuracy

Target met against benchmarks

5 Key Metrics

Comprehensive performance evaluation

Fastest Runtime

Efficient execution times

Robust Development Practices

Ensuring Maintainability and Clarity in Code

Achievements

Key Milestones in Development

Zero-Dependency ML

Modular Architecture

Integrated Solutions

Thank you!